



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
"МИРЭА - Российский технологический университет"

РТУ МИРЭА

Институт информационных технологий (ИТ)
Кафедра инструментального и прикладного программного обеспечения

**ОТЧЕТ
ПО ПРАКТИЧЕСКОЙ РАБОТЕ №3
по дисциплине
«Технологии виртуализации клиент-серверных приложений»**

Выполнил студент группы ИКБО-16-22

Грибков А.С.

Принял преподаватель кафедры ИиППО

Волков М.Ю.

Практические работы выполнены

«__»_____2025 г.

«Зачтено»

«__»_____2025 г.

СОДЕРЖАНИЕ

1 ЦЕЛЬ РАБОТЫ.....	3
2 ХОД РАБОТЫ	4
2.1 Задание 1. Создание Dockerfile с демонстрацией всех команд.....	4
2.2 Задание 2. Веб-приложение Spring Boot с PostgreSQL	8
3 ВЫВОД.....	13
4 ОТВЕТЫ НА ВОПРОСЫ К ПРАКТИЧЕСКОЙ РАБОТЕ	14
4.1 Опишите процесс запуска приложения внутри контейнера Linux, используя Docker	14
4.2 Что такое образ Docker и для чего он нужен?	15
4.3 Как соотносятся между собой файлы Dockerfile и Docker-Compose?.....	15
4.4 Что такое Dockerfile?.....	16
4.5 Опишите политики перезапуска контейнера	17
4.6 Назовите все возможные состояния контейнеров.....	18
5 СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ.....	19

1 ЦЕЛЬ РАБОТЫ

Целью данной практической работы является изучение основ работы с Docker и создание Docker-образов с использованием Dockerfile. Docker представляет собой программную платформу для разработки, доставки и запуска контейнерных приложений, позволяющую создавать контейнеры, автоматизировать их запуск и развертывание, управлять жизненным циклом.

Основные задачи работы:

Изучить основные команды Dockerfile и создать образ, демонстрирующий использование всех требуемых директив.

Разработать веб-приложение на Spring Boot с интеграцией PostgreSQL и создать для него многоэтапный Dockerfile.

Освоить принципы многоэтапной сборки Docker-образов для оптимизации размера итогового образа.

Настроить оркестрацию контейнеров с помощью Docker Compose.

Загрузить созданный Docker-образ в публичный реестр DockerHub.

2 ХОД РАБОТЫ

2.1 Задание 1. Создание Dockerfile с демонстрацией всех команд

В рамках первого задания требуется создать один или несколько Dockerfile, в которых каждая из 13 указанных команд Docker используется хотя бы один раз. Был создан Dockerfile для Node.js приложения, демонстрирующий использование всех требуемых директив.

Листинг 2.1 — Dockerfile с использованием всех 13 команд Docker

```
FROM node:18-slim
LABEL maintainer="Student Name student@example.com" LABEL
version="1.0" LABEL description="Dockerfile demonstrating all
Docker commands for Task 1"
ENV NODE_ENV=production ENV PORT=3000 ENV APP_NAME="docker-
commands-demo"
WORKDIR /app
RUN groupadd -r appuser && useradd -r -g appuser appuser
COPY package.json ./
RUN apt-get update &&
apt-get install -y wget &&
npm install -production &&
mkdir -p /app/data /app/downloads &&
apt-get clean &&
rm -rf /var/lib/apt/lists/*
ADD https://raw.githubusercontent.com/nodejs/node/main/README.md
/app/downloads/node-readme.md
COPY . .
RUN chown -R appuser:appuser /app
USER appuser
VOLUME ["/app/data"]
EXPOSE $PORT
ONBUILD COPY . /app ONBUILD RUN npm install
ENTRYPOINT ["node"]
CMD ["app.js"]
```

В созданном Dockerfile использованы все 13 требуемых команд:

FROM определяет базовый образ node:18-slim, на основе которого будет создан новый образ.

`LABEL` добавляет метаданные к образу, включая информацию о разработчике, версии и описании.

`ENV` устанавливает переменные окружения для контейнера, такие как `NODE_ENV`, `PORT` и `APP_NAME`.

`WORKDIR` устанавливает рабочую директорию для последующих инструкций в `/app`.

`RUN` выполняет команды во время сборки образа, включая создание пользователя, установку зависимостей и настройку прав доступа.

`COPY` копирует файлы из контекста сборки в образ, включая `package.json` и исходный код приложения.

`ADD` загружает файл `README.md` из внешнего URL и добавляет его в образ.

`USER` переключает контекст выполнения на непривилегированного пользователя `appuser` для повышения безопасности.

`VOLUME` создает точку монтирования для постоянного хранения данных в директории `/app/data`.

`EXPOSE` документирует порт 3000, на котором приложение принимает подключения.

`ONBUILD` добавляет триггерные инструкции, которые будут выполнены при использовании данного образа как базового для других образов.

`ENTRYPOINT` определяет исполняемый файл `node` как основную точку входа в контейнер.

`CMD` предоставляет аргументы по умолчанию для `ENTRYPOINT`, указывая запуск файла `app.js`.

Для сборки Docker-образа выполнена команда `docker build` с указанием тега образа.

```
Sending build context to Docker daemon 11.78kB
Step 1/21 : FROM node:18-slim
----> f9ab18e354e6
Step 2/21 : LABEL maintainer="Student Alexander Gribkov <gribkov.a.s@edu.mirea.ru>"
----> Running in 4861c4af1d26
----> Removed intermediate container 4861c4af1d26
----> dfb6cded6469
Step 3/21 : LABEL version="1.0"
----> Running in 7eef10b5875d
----> Removed intermediate container 7eef10b5875d
----> 869937b5da81
Step 4/21 : LABEL description="Dockerfile demonstrating all Docker commands for Task 1"
----> Running in c13fb85c14c6
----> Removed intermediate container c13fb85c14c6
----> db65e035cd3f
Step 5/21 : ENV NODE_ENV=production
----> Running in 18d7b02008fd
----> Removed intermediate container 18d7b02008fd
----> ee7833a10020
Step 6/21 : ENV PORT=3000
----> Running in 76dbca486cfe
----> Removed intermediate container 76dbca486cfe
----> 3de0148c3928
Step 7/21 : ENV APP_NAME="docker-commands-demo"
----> Running in 34fccd30b6f8
----> Removed intermediate container 34fccd30b6f8
----> e476734fbdfa
Step 8/21 : WORKDIR /app
----> Running in afb3e885c352
----> Removed intermediate container afb3e885c352
----> ecc38e416c62
```

Рисунок 2.1 — Сборка Docker образа для задания 1

После успешной сборки образ запущен в контейнере с пробросом порта 3000 на хост-машину.

```

----> ee7833a10020
Step 6/21 : ENV PORT=3000
----> Running in 76dbca486cfe
----> Removed intermediate container 76dbca486cfe
----> 3de0148c3928
Step 7/21 : ENV APP_NAME="docker-commands-demo"
----> Running in 34fccd30b6f8
----> Removed intermediate container 34fccd30b6f8
----> e476734fbdfa
Step 8/21 : WORKDIR /app
----> Running in afb3e885c352
----> Removed intermediate container afb3e885c352
----> ecc38e416c62
Step 9/21 : RUN groupadd -r appuser && useradd -r -g appuser appuser
----> Running in 05b9b7280860
----> Removed intermediate container 05b9b7280860
----> b375de6755c5
Step 10/21 : COPY package.json ./
----> 7f53f7cbcf38
Step 11/21 : RUN apt-get update && apt-get install -y wget && npm install --production && mkdir -p /a
pp/data /app/downloads && apt-get clean && rm -rf /var/lib/apt/lists/*
----> Running in f14097376615
Get:1 http://deb.debian.org/debian bookworm InRelease [151 kB]
Get:2 http://deb.debian.org/debian bookworm-updates InRelease [55.4 kB]
Get:3 http://deb.debian.org/debian-security bookworm-security InRelease [48.0 kB]
Get:4 http://deb.debian.org/debian bookworm/main arm64 Packages [8693 kB]
Get:5 http://deb.debian.org/debian bookworm-updates/main arm64 Packages [6936 B]
Get:6 http://deb.debian.org/debian-security bookworm-security/main arm64 Packages [276 kB]
Fetched 9230 kB in 5s (1845 kB/s)
Reading package lists...
Reading package lists...
Building dependency tree...
Reading state information...
The following additional packages will be installed:
  ca-certificates libpsl5 libssl3 openssl publicsuffix
The following NEW packages will be installed:
  ca-certificates libpsl5 libssl3 openssl publicsuffix wget
0 upgraded, 6 newly installed, 0 to remove and 17 not upgraded.

```

Рисунок 2.2 — Запуск контейнера задания 1

Для проверки работоспособности приложения выполнены запросы к реализованным эндпоинтам. Приложение предоставляет информацию о используемых Docker-командах, состояние здоровья сервиса и список загруженных файлов.

```
✓ Testing endpoints...
Testing root endpoint:
{
  "message": "Welcome to docker-commands-demo!",
  "environment": "production",
  "port": "3000",
  "dockerCommands": [
    "FROM",
    "RUN",
    "LABEL",
    "CMD",
    "EXPOSE",
    "ENV",
    "ADD",
    "COPY",
    "ENTRYPOINT",
    "VOLUME",
    "USER",
    "WORKDIR",
    "ONBUILD"
  ]
}

Testing health endpoint:
{
  "status": "healthy",
  "timestamp": "2025-10-12T12:03:48.252Z"
}

Testing files endpoint:
{
  "downloads": [
    "node-readme.md"
  ],
  "dataVolume": "mounted"
}

✓ All tests completed!
```

Рисунок 2.3 — Тестирование эндпоинтов приложения задания 1

2.2 Задание 2. Веб-приложение Spring Boot с PostgreSQL

Второе задание требует создания Dockerfile для веб-приложения, написанного с использованием фреймворка Spring Boot и взаимодействующего

с СУБД PostgreSQL. Dockerfile должен содержать две изолированные стадии: сборку и запуск jar файла.

Приложение реализует три эндпоинта: добавление элемента, вывод списка всех элементов и получение фото герба РТУ МИРЭА. При сборке проекта с помощью утилиты `wget` загружается файл изображения герба МИРЭА, который затем становится доступным по соответствующему эндпоинту веб-сервиса.

Листинг 2.2 — Многоэтапный Dockerfile для Spring Boot приложения

```
FROM gradle:8.14-jdk21 AS build
LABEL maintainer="Александр Грибков student@mirea.ru" LABEL
version="1.0" LABEL description="Multi-stage Spring Boot
application with PostgreSQL"
WORKDIR /app
COPY build.gradle.kts ./ COPY src src
RUN apt-get update &&
apt-get install -y wget &&
mkdir -p src/main/resources/static &&
wget -O src/main/resources/static/MIREA_Gerb_Colour.png
https://www.mirea.ru/upload/medialibrary/80f/MIREA_Gerb_Colour.png
&&
apt-get clean &&
rm -rf /var/lib/apt/lists/*
RUN gradle build -no-daemon -x test
FROM eclipse-temurin:21-jre
WORKDIR /app
ENV DB_HOST=postgres ENV DB_PORT=5432 ENV DB_NAME=mirea_db ENV
DB_USERNAME=mirea_user ENV DB_PASSWORD=mirea_password
COPY --from=build /app/build/libs/*.jar app.jar COPY --from=build
/app/src/main/resources/static/MIREA_Gerb_Colour.png /app/static/
EXPOSE 8080
ONBUILD RUN echo "Сборка и запуск произведены. Автор: Александр
Грибков"
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Dockerfile использует многоэтапную сборку для оптимизации размера итогового образа. Первая стадия `build` использует образ `gradle:8.14-jdk21` для компиляции приложения и загрузки герба МИРЭА. Вторая стадия использует минимальный образ `eclipse-temurin:21-jre`, содержащий только среду выполнения Java, и копирует из первой стадии только скомпилированный jar-файл и статические ресурсы.

Переменные окружения для подключения к PostgreSQL определены в Dockerfile через директиву `ENV`. Порт базы данных получается из переменной окружения `DB_PORT`, что соответствует требованиям задания.

В LABEL указаны ФИО и группа студента. Команда ONBUILD выводит сообщение с информацией об авторе при использовании данного образа как базового для других образов.

Для оркестрации контейнеров создан файл docker-compose.yml, который описывает два сервиса: PostgreSQL базу данных и Spring Boot приложение.

Листинг 2.3 — Файл docker-compose.yml для оркестрации контейнеров

```
services:
  postgres:
    image: postgres:15-alpine
    container_name: mirea-postgres
    environment:
      POSTGRES_DB: mirea_db
      POSTGRES_USER: mirea_user
      POSTGRES_PASSWORD: mirea_password
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - mirea-network

  mirea-app:
    build: .
    container_name: mirea-spring-app
    ports:
      - "8080:8080"
    environment:
      DB_HOST: postgres
      DB_PORT: 5432
      DB_NAME: mirea_db
      DB_USERNAME: mirea_user
      DB_PASSWORD: mirea_password
    depends_on:
      - postgres
    networks:
      - mirea-network

volumes:
  postgres_data:

networks:
  mirea-network:
    driver: bridge
```

Файл docker-compose.yml определяет сервис postgres на основе официального образа PostgreSQL 15 Alpine и сервис mirea-app, который собирается из локального Dockerfile. Оба сервиса подключены к общей сети

mirea-network типа bridge, что обеспечивает их взаимодействие. Для PostgreSQL создан именованный том postgres_data для постоянного хранения данных.

Выполнена сборка многоэтапного Docker-образа с помощью Docker Compose.

```
[+] Building 145.9s (17/17) FINISHED                                docker:colima
=> [mirea-app internal] load build definition from Dockerfile        0.0s
=> => transferring dockerfile: 1.10kB                                0.0s
=> [mirea-app internal] load metadata for docker.io/library/eclipse-temurin:21-jre 1.6s
=> [mirea-app internal] load metadata for docker.io/library/gradle:8.14-jdk21 1.7s
=> [mirea-app internal] load .dockerignore                            0.0s
=> => transferring context: 2B                                         0.0s
=> [mirea-app build 1/6] FROM docker.io/library/gradle:8.14-jdk21@sha256:f45a7054d73c0784193b87150537cff6 0.0s
=> => resolve docker.io/library/gradle:8.14-jdk21@sha256:f45a7054d73c0784193b87150537cff692ca825e5d2778be 0.0s
=> [mirea-app stage-1 1/4] FROM docker.io/library/eclipse-temurin:21-jre@sha256:e48ae1e7797cee055b4392e61 0.0s
=> => resolve docker.io/library/eclipse-temurin:21-jre@sha256:e48ae1e7797cee055b4392e619c19a6653a018a47ab 0.0s
=> [mirea-app internal] load build context                            0.0s
=> => transferring context: 923B                                       0.0s
```

Рисунок 2.4 — Сборка образа через Docker Compose

После сборки запущены контейнеры с помощью команды docker-compose up. Приложение успешно подключилось к базе данных PostgreSQL и начало обрабатывать HTTP-запросы.

```
[+] Running 5/5
✔ mirea-app Built 0.0s
✔ Network task-2-spring-boot-app_mirea-network Created 0.1s
✔ Volume task-2-spring-boot-app_postgres_data Created 0.0s
✔ Container mirea-postgres Started 0.5s
✔ Container mirea-spring-app Started 0.7s
```

Рисунок 2.5 — Запуск контейнеров через Docker Compose

Проверена работоспособность эндпоинта добавления элемента. Выполнен POST-запрос к /api/items с JSON-данными, содержащими название и описание элемента.

```
2. Testing POST /api/items (adding new item):
{
  "id": 1,
  "name": "Test Item",
  "description": "Test Description"
}
```

Рисунок 2.6 — Тестирование POST запроса добавления элемента

Проверена работоспособность эндпоинта получения списка всех элементов. Выполнен GET-запрос к /api/items, который возвращает JSON-массив с добавленными элементами.

```
3. Testing GET /api/items (should show the added item):  
[  
  {  
    "id": 1,  
    "name": "Test Item",  
    "description": "Test Description"  
  }  
]
```

Рисунок 2.7 — Тестирование GET запроса списка элементов

Проверена работоспособность эндпоинта получения герба МИРЭА. Выполнен GET-запрос к /api/mirea-logo, который возвращает PNG-изображение, загруженное на этапе сборки образа.

```
4. Testing GET /api/mirea-logo (MIREA emblem):  
✓ MIREA logo endpoint working (PNG image detected)
```

Рисунок 2.8 — Тестирование GET запроса герба МИРЭА

Проверено подключение к контейнеру PostgreSQL и наличие созданных таблиц. С помощью команды `docker exec` выполнен вход в контейнер базы данных и запрос к таблице `items`, подтверждающий сохранение данных.

```
PostgreSQL logs:  
server stopped  
  
PostgreSQL init process complete; ready for start up.  
  
2025-10-12 12:15:12.867 UTC [1] LOG:  starting PostgreSQL 15.14 on aarch64-unknown-linux-musl, compiled by gcc (Alpine 14.2.0) 14.2.0, 64-bit  
2025-10-12 12:15:12.867 UTC [1] LOG:  listening on IPv4 address "0.0.0.0", port 5432  
2025-10-12 12:15:12.867 UTC [1] LOG:  listening on IPv6 address ":::", port 5432  
2025-10-12 12:15:12.871 UTC [1] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"  
2025-10-12 12:15:12.882 UTC [75] LOG:  database system was shut down at 2025-10-12 12:15:12 UTC  
2025-10-12 12:15:12.889 UTC [1] LOG:  database system is ready to accept connections
```

Рисунок 2.9 — Проверка данных в PostgreSQL

Созданный Docker-образ загружен в публичный реестр DockerHub для обеспечения возможности его использования на других машинах.

3 ВЫВОД

В результате выполнения практической работы были успешно освоены основные принципы работы с Docker и создания Docker-образов с использованием Dockerfile. Создан Dockerfile, демонстрирующий использование всех 13 требуемых команд Docker, включая FROM, RUN, LABEL, CMD, EXPOSE, ENV, ADD, COPY, ENTRYPOINT, VOLUME, USER, WORKDIR и ONBUILD.

Разработано веб-приложение на Spring Boot с интеграцией PostgreSQL, реализующее три функциональных эндпоинта для работы с элементами и отдачи статических ресурсов. Создан многоэтапный Dockerfile, использующий изолированные стадии сборки и выполнения, что позволило значительно уменьшить размер итогового образа за счет исключения инструментов разработки из финального образа.

Освоены принципы оркестрации контейнеров с помощью Docker Compose. Создана конфигурация, описывающая взаимодействие между контейнером приложения и контейнером базы данных через изолированную виртуальную сеть. Настроено постоянное хранение данных PostgreSQL с использованием Docker-томов.

Проверена работоспособность всех реализованных эндпоинтов, включая добавление элементов в базу данных, получение списка элементов и отдачу статического изображения герба МИРЭА, загруженного на этапе сборки образа с помощью утилиты wget. Подтверждено корректное взаимодействие приложения с базой данных PostgreSQL через переменные окружения.

Созданный Docker-образ успешно загружен в публичный реестр DockerHub, что обеспечивает возможность его использования на других машинах без необходимости повторной сборки. Полученные навыки работы с Docker являются важными для современной разработки приложений, обеспечивая их портируемость, изоляцию и упрощение развертывания.

4 ОТВЕТЫ НА ВОПРОСЫ К ПРАКТИЧЕСКОЙ РАБОТЕ

4.1 Опишите процесс запуска приложения внутри контейнера Linux, используя Docker

Процесс запуска приложения внутри контейнера Linux с использованием Docker начинается с создания Dockerfile, который содержит инструкции для сборки образа. Dockerfile определяет базовый образ с помощью директивы FROM, устанавливает зависимости через RUN, копирует файлы приложения с помощью COPY или ADD, и определяет команду запуска через ENTRYPOINT или CMD.

После создания Dockerfile выполняется команда `docker build` для создания образа. Docker читает инструкции из Dockerfile и выполняет каждую из них последовательно, создавая слои образа. Каждая инструкция RUN, COPY и ADD создает новый слой, который представляет собой дельту изменений по сравнению с предыдущим слоем. Промежуточные слои кэшируются для ускорения последующих сборок.

Когда образ создан, контейнер запускается с помощью команды `docker run`. Docker создает изолированное окружение на основе образа, включающее собственную файловую систему, сетевое пространство имен и пространство имен процессов. Контейнер получает копию файловой системы образа с дополнительным записываемым слоем поверх слоев образа, доступных только для чтения.

При запуске контейнера Docker выполняет команду, определенную в ENTRYPOINT и CMD. Приложение запускается внутри изолированного окружения контейнера с заданными переменными окружения, сетевыми настройками и ограничениями ресурсов. Контейнер продолжает работать до тех

пор, пока выполняется основной процесс, определенный в ENTRYPOINT или CMD.

4.2 Что такое образ Docker и для чего он нужен?

Docker-образ представляет собой шаблон для создания Docker-контейнеров. Это исполняемый пакет, содержащий все необходимое для запуска приложения: код, среду выполнения, библиотеки, переменные окружения и файлы конфигурации. Образ является неизменяемым и служит основой для создания множества идентичных контейнеров.

Docker-образ состоит из слоев, где каждое изменение записывается в новый слой. Такая слоистая архитектура обеспечивает эффективное использование дискового пространства и ускорение передачи образов. При загрузке или скачивании Docker-образа операции производятся только с теми слоями, которые были изменены. Слои исходного Docker-образа являются общими между всеми его версиями и не дублируются.

Образы Docker нужны для обеспечения портируемости приложений. Образ гарантирует, что приложение будет запускаться одинаково в любом окружении, имеющем Docker. Это решает проблему зависимостей и конфликтов версий библиотек, так как все зависимости упакованы внутри образа. Образы также упрощают развертывание приложений, позволяя быстро создавать новые экземпляры контейнеров на основе готового образа.

4.3 Как соотносятся между собой файлы Dockerfile и Docker-Compose?

Dockerfile и Docker Compose выполняют разные, но дополняющие друг друга функции в экосистеме Docker. Dockerfile представляет собой текстовый

файл с инструкциями для создания одного Docker-образа. Он описывает, как собрать образ: какой базовый образ использовать, какие файлы копировать, какие команды выполнить и как запустить приложение.

Docker Compose представляет собой инструмент для определения и запуска многоконтейнерных Docker-приложений. Файл `docker-compose.yml` описывает конфигурацию нескольких сервисов, их взаимодействие, сетевые настройки и тома для хранения данных. Docker Compose использует образы, которые могут быть созданы из Dockerfile или загружены из реестра.

Связь между Dockerfile и Docker Compose заключается в том, что Docker Compose может ссылаться на Dockerfile для сборки образа определенного сервиса. В секции `build` файла `docker-compose.yml` указывается путь к директории с Dockerfile, и при выполнении `docker-compose up` образ собирается автоматически. Таким образом, Dockerfile описывает как создать один образ, а Docker Compose определяет как запустить и организовать взаимодействие нескольких контейнеров на основе этих образов.

4.4 Что такое Dockerfile?

Dockerfile представляет собой текстовый документ, который содержит все команды, необходимые для сборки Docker-образа. Это своего рода сценарий сборки, в котором последовательно перечислены инструкции для создания образа. Docker автоматически создает образы, читая инструкции из Dockerfile.

Dockerfile придерживается определенного формата и набора инструкций. Каждая инструкция в Dockerfile создает слой в образе. Основные инструкции включают `FROM` для определения базового образа, `RUN` для выполнения команд во время сборки, `COPY` и `ADD` для копирования файлов, `WORKDIR` для установки рабочей директории, `ENV` для определения переменных окружения, `EXPOSE` для документирования портов, и `CMD` или `ENTRYPOINT` для указания команды запуска контейнера.

Dockerfile обеспечивает воспроизводимость сборки образов. Один и тот же Dockerfile при сборке создаст идентичный образ независимо от окружения, в котором выполняется сборка. Dockerfile также служит документацией для образа, ясно показывая, какие зависимости установлены, какие файлы включены и как настроено приложение.

4.5 Опишите политики перезапуска контейнера

Docker предоставляет несколько политик перезапуска контейнеров, которые определяют поведение системы при остановке контейнера. Политика перезапуска задается с помощью флага `--restart` при запуске контейнера командой `docker run`.

Политика `no` является значением по умолчанию и означает, что контейнер не будет автоматически перезапущен ни при каких обстоятельствах. Это подходит для контейнеров, которые выполняют разовые задачи или когда требуется полный контроль над жизненным циклом контейнера.

Политика `on-failure` перезапускает контейнер только если процесс завершился с ненулевым кодом выхода, что указывает на ошибку. Можно дополнительно указать максимальное количество попыток перезапуска, например `on-failure:5`. Это полезно для приложений, которые могут временно сталкиваться с проблемами, но должны останавливаться при постоянных сбоях.

Политика `always` указывает Docker всегда перезапускать контейнер, независимо от кода выхода. Если контейнер остановлен вручную, он будет перезапущен только после перезапуска демона Docker или ручного перезапуска контейнера. Эта политика подходит для сервисов, которые должны быть постоянно доступны.

Политика `unless-stopped` аналогична `always`, но с одним отличием: если контейнер был остановлен вручную, он не будет автоматически перезапущен после перезагрузки демона Docker. Это позволяет администратору явно

контролировать, какие контейнеры должны запускаться после перезагрузки системы.

4.6 Назовите все возможные состояния контейнеров

Контейнер Docker может находиться в одном из шести возможных состояний, каждое из которых отражает текущий этап его жизненного цикла.

Состояние Created указывает, что контейнер создан, но еще не запущен. В этом состоянии все ресурсы для контейнера выделены, но процесс внутри контейнера не выполняется.

Состояние Running означает, что контейнер активно работает и процесс внутри контейнера выполняется. Это основное рабочее состояние контейнера, в котором приложение обрабатывает запросы и выполняет свои функции.

Состояние Restarting указывает, что контейнер находится в процессе перезапуска. Это временное состояние, которое возникает при применении политики автоматического перезапуска или при ручном перезапуске контейнера.

Состояние Paused означает, что контейнер приостановлен. В этом состоянии все процессы контейнера заморожены, но их состояние сохранено в памяти. Контейнер может быть возобновлен командой `docker unpause` без потери состояния.

Состояние Exited указывает, что контейнер закончил свою работу и остановлен. Процесс внутри контейнера завершился, но файловая система контейнера и его конфигурация сохранены. Контейнер может быть запущен снова командой `docker start`.

Состояние Dead означает контейнер, который демон Docker попытался остановить, но не смог этого сделать. Это аварийное состояние, которое обычно указывает на проблемы с ресурсами или с самим Docker.

5 СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. 50 вопросов по Docker, которые задают на собеседованиях, и ответы на них. Текст : электронный. habr.com : сайт. URL: <https://habr.com/ru/company/southbridge/blog/528206/>
2. Docker Documentation. Текст : электронный. docs.docker.com : сайт. URL: <https://docs.docker.com/>
3. Dockerfile reference. Docker Documentation. Текст : электронный. docs.docker.com : сайт. URL: <https://docs.docker.com/engine/reference/builder/>
4. Что такое режим Docker Swarm и когда его использовать? CloudSavvy ИТ. Текст : электронный. cpab.ru : сайт. URL: <https://cpab.ru/chto-takoe-rezhim-docker-swarm-i-kogda-ego-ispolzovat-cloudsavvy-it/>
5. Best practices for writing Dockerfiles. Docker Documentation. Текст : электронный. docs.docker.com : сайт. URL: <https://docs.docker.com/develop/dev-best-practices/>
6. Use multi-stage builds. Docker Documentation. Текст : электронный. docs.docker.com : сайт. URL: <https://docs.docker.com/build/building/multi-stage/>